

Relatorio de Auditoria de Seguranca - NexoDoc

Data	31 de agosto de 2026
Snapshot	Auditoria original + atualizacoes P1/P2 no worktree; HEAD c440424; 884 commits consultados
Escopo	Next.js, 54 arquivos de rota/74 handlers, Prisma/PostgreSQL, Auth.js, frontend React, Docker/Render, scripts, documentacao, historico Git e bundle .next/static
Fora do escopo	Teste de invasao em producao, validacao online da chave encontrada e revisao de dependencias/CVEs

Nota metodologica

A categoria 'banco sem tranca' foi mapeada para o mecanismo real do NexoDoc: membership em OrganizationMember e filtros Prisma por Project.organizationId. 'Permissao no navegador' foi verificada cruzando gates isAdmin/orgRole com cada endpoint correspondente. IDOR cobriu todos os handlers e IDs em path, query e body. Segredos cobriram HEAD, configuracoes de deploy, historico Git e bundle do cliente. XSS cobriu sinks React/DOM, Markdown/HTML, URLs e HTML de e-mail.

Este documento preserva os sete achados do snapshot original e registra separadamente o estado de remediacao P1/P2. Condicoes como feature flag, janela de status ou chave ainda ativa aparecem explicitamente em cada achado.

1. Stack detectada e modelo de seguranca

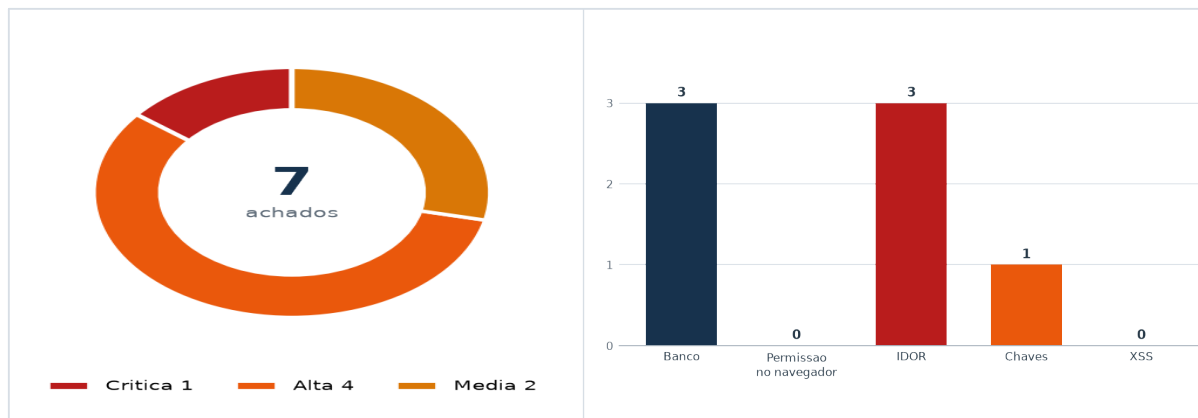
Camada	Tecnologia detectada	Como a auditoria adaptou a categoria
Linguagem	TypeScript/JavaScript; Node.js 22	Análise de handlers App Router, helpers server-side e scripts
Framework	Next.js 16.2.6 + React 19.2.6	Rotas em app/api/**/route.ts; SSR e componentes client
ORM/banco	Prisma 7.8 + PostgreSQL via pg/PrismaPg	Sem RLS; isolamento manual por OrganizationMember e Project.organizationId
Auth	Auth.js/NextAuth beta 31; Google OAuth; Credentials apenas em dev	requireActor para tenant; requirePlatformAdmin + token para admin
Frontend	React, Tailwind CSS 4, App Router	Gates isAdmin/orgRole e sinks DOM/URL/Markdown
Deploy	Docker multi-stage e Render Blueprint; conversor Docker legado	Segredos em env sync:false, startup/migrations e imagem final
Ausente	Sem GitHub Actions, Helm, Terraform ou docker-compose no repositório	Categorias explicitamente marcadas como não aplicáveis

Mecanismo de isolamento identificado

O NexoDoc não usa Supabase/RLS. A autorização é feita no servidor: requireActor resolve a sessão, a conta ativa é um OrganizationMember ACTIVE; o actor recebe organizationId. Queries corretas usam esse ID diretamente, assertProjectAccess ou auditWhereForActor/auditByIdWhereForActor. O P2 também tornou organizationId obrigatório em AuditLearning. F-01 continua aberto; F-02 e F-05 foram corrigidos.

2. Resumo executivo

Foram confirmados 7 achados no snapshot original: 1 critico, 4 altos e 2 medios. As atualizacoes P1/P2 fecharam F-02 a F-06; a migracao de F-05 foi aplicada e provada no banco de desenvolvimento. F-07 foi mitigado no HEAD, mas depende de revogacao e limpeza do historico remoto. Permanece aberto F-01.



CRITICA	1	Entrada externa no tenant padrao
ALTA	4	Historico global, chat/IDOR, endpoints auxiliares e chave
MEDIA	2	Learnings globais e relacoes de artefato
BAIXA	0	Nenhum achado verificado

Pontos fracos centrais

1) o cadastro automatico ainda transforma autenticao Google em autorizacao ao tenant; 2) o incidente da chave exige revogacao e saneamento do historico fora deste worktree; 3) os aprendizados ja estao isolados, mas regra de papel e autoria permanecem como hardening de governanca.

Pontos fortes centrais

A aplicacao tem blocos reutilizaveis: requireActor, assertProjectAccess, checkAdminRequest, auditWhereForActor/auditByIdWhereForActor e InvalidArtifactRelation. P1/P2 adicionaram provas cruzadas para auditorias, aprendizados e artefatos, alem da varredura local contra segredos hardcoded.

Estado de remediacao

Achado	Estado atual
F-01	ABERTO - depende da política de entrada Google: convite, allowlist ou dominio autorizado.
F-02	CORRIGIDO NO WORKTREE - recent e quality usam o escopo canonico do ator; prova end-to-end com dois escritorios passou.
F-03	CORRIGIDO NO WORKTREE - chat valida o auditId antes de memoria/update e deriva o projectId da auditoria autorizada.
F-04	CORRIGIDO NO WORKTREE - cancelamento, feedback e delta aplicam o mesmo predicado de posse por auditoria.
F-05	CORRIGIDO NO WORKTREE E NO BANCO DEV - organizationId e obrigatorio, o legado foi migrado para org-prosul e todas as operacoes usam o tenant. Regra de papel e autoria continuam como hardening.
F-06	CORRIGIDO NO WORKTREE - auditId e IdDraftId sao validados contra o mesmo projectId antes da criacao; prova cruzada passou.
F-07	PARCIAL - literal removido e scanner local adicionado; revogacao e limpeza do historico remoto continuam pendentes.

3. Tabela consolidada de achados

ID	Severidade	Categoria	Arquivo:linha	Descricao
F-01	CRITICA	Banco sem tranca	auth.ts:11-36; lib/access-control.ts:121-122,255-264,274-279	Qualquer conta Google vira membro ativo da organizacao padrao
F-02	ALTA	Banco sem tranca	app/api/audits/recent/route.ts:10-14,58-85; audits/quality:31-34	Historico recente e qualidade agregam auditorias de todas as organizacoes
F-03	ALTA	IDOR	app/api/audit/chat/route.ts:114-125,131-141,168-183,198-213; memoria:104-113	Chat aceita auditId arbitrario, le o texto integral e sobrescreve o parecer
F-04	ALTA	IDOR	audit/[id]/cancel:58-64; audits/[id]/feedback:68-71,234-308; audit/delta:63-65	Endpoints auxiliares de auditoria usam ID sem validar organizacao
F-05	MEDIA	Banco sem tranca	prisma/schema.prisma:698-717; api/learnings:49-88; audit-learnings.ts:205-337	AuditLearning e global e pode ser alterado por membro de qualquer organizacao
F-06	MEDIA	IDOR	projects/[id]/artifacts:167-190; lib/project-store.ts:231-248	Criacao de artefato aceita auditId e IdDraftId de outro projeto ou tenant
F-07	ALTA	Chaves expostas	scripts/test-ai-precos.ts:90-94; Git 311196f	Chave OpenAI real foi copiada para teste e permanece no Git

Categorias sem achado

Permissao definida no navegador: nenhum caso confirmado. O gate isAdmin das paginas/ataltos administrativos tem verificacao equivalente no backend. As operacoes de convite e cadastro manual de projeto tambem verificam papel no servidor.

Inputs sem tratamento (XSS): nenhum caso confirmado. Nao existe sink DOM/HTML de producao; a aplicacao nao usa biblioteca de sanitizacao porque nao renderiza HTML/Markdown nao confiavel. Os templates de e-mail usam funcoes de escape nos campos dinamicos.

4. Achados detalhados

As evidencias abaixo preservam o codigo observado no snapshot original. O callout de cada cartao informa se a falha foi corrigida, esta parcial ou continua aberta no worktree atual.

F-01 - Qualquer conta Google vira membro ativo da organizacao padrao

CRITICA

Categoria: Banco sem tranca. O login Google nao aplica allowlist de dominio ou e-mail. No primeiro acesso, getUserAccess chama garantirEscritorioPadrao; se NEXODOC_ESCRITORIO_PADRAO estiver ausente, o codigo escolhe org-prosul e cria um vinculo MEMBER/ACTIVE. O render.yaml atual nao declara a variavel, portanto o caminho padrao esta ativo no deploy descrito pelo repositorio. Depois disso, o novo membro passa pelo mesmo isolamento por organizacao usado para projetos e auditorias.

Status de remediacao em 31/08/2026: ABERTO - depende da politica de entrada Google: convite, allowlist ou dominio autorizado.

Condicao de explorabilidade: Exploravel quando a aplicacao e o OAuth Google sao acessiveis externamente e NEXODOC_ESCRITORIO_PADRAO esta ausente. Definir a variavel como string vazia desliga o comportamento.

Evidencia - auth.ts:11-36

```
11: export const { auth, handlers, signIn, signOut } = NextAuth({
12:   providers: [
13:     Google({ ... }),
20:     ...(isDevAuthEnabled() ? [Credentials(...)] : []),
36: ],
```

Evidencia - lib/access-control.ts:121-122 255-264 274-279

```
121: await ativarConvitePendente(user.id, normalizedEmail);
122: await garantirEscritorioPadrao(user.id, normalizedEmail, name);
255: await prisma.organizationMember.upsert({
256:   where: { organizationId_email: { organizationId, email } },
257:   create: { organizationId, email, name: ..., userId,
262:     role: "MEMBER",
263:     status: "ACTIVE" },
274:   function escritorioPadrao() {
275:     const bruto = process.env.NEXODOC_ESCRITORIO_PADRAO;
277:     if (bruto === undefined) {
278:       return "org-prosul";
```

F-02 - Historico recente e qualidade agregam auditorias de todas as organizacoes

ALTA

Categoria: Banco sem tranca. Depois de requireActor, a listagem executa audit.findMany sem where de project.organizationId e devolve report, result, nomes de arquivos e IDs. O endpoint de qualidade faz groupBy global de AuditFeedback. Isso ignora o mecanismo canonico do projeto, que e membership + organizationId. Em producao os dois dependem de NEXODOC_ENABLE_PUBLIC_AUDIT_HISTORY=true; fora de producao ficam ativos automaticamente.

Status de remediacao em 31/08/2026: CORRIGIDO NO WORKTREE - recent e quality usam o escopo canonico do ator; prova end-to-end com dois escritorios passou.

Condicao de explorabilidade: Exploravel em qualquer ambiente nao-production, ou em producao quando NEXODOC_ENABLE_PUBLIC_AUDIT_HISTORY=true. Exige ao menos duas organizacoes com dados, ou dados legados de pessoas diferentes.

Evidencia - app/api/audits/recent/route.ts:10-14 58-85

```
10: function isRecentHistoryEnabled() {
11:   return (
12:     process.env.NODE_ENV !== "production" ||
13:     process.env.NEXODOC_ENABLE_PUBLIC_AUDIT_HISTORY === "true"
14:   );
58: const audits = await getPrisma().audit.findMany({
59:   take: getLimit(request),
60:   orderBy: { createdAt: "desc" },
63:   select: { id: true, ... result: true, report: true, ... files: {...} }
```

```
85: });
```

Evidencia - app/api/audits/quality/route.ts:31-34

```
31: const grouped = await getPrisma().auditFeedback.groupBy({
32:   by: ["verdict"],
33:   _count: { _all: true },
34: });
```

F-03 - Chat aceita auditId arbitrario, le o texto integral e sobrescreve o parecer

ALTA

Categoria: IDOR. A rota valida apenas que existe um ator. O auditId vem do corpo, e carregarMemoriaDoDocumento consulta AuditText somente por auditId. O texto integral do memorial entra nas ferramentas do modelo; ao registrar um achado, gravarAchadoNoParecer atualiza Audit pelo mesmo ID, sem posse ou organizationId. O projectId do historico tambem vem do cliente. Um membro que conheca o ID de outra auditoria pode consultar seu conteudo por linguagem natural e alterar o relatorio persistido.

Status de remediacao em 31/08/2026: CORRIGIDO NO WORKTREE - chat valida o auditId antes de memoria/update e deriva o projectId da auditoria autorizada.

Condicao de explorabilidade: Exige uma sessao de membro ativo e conhecimento de um auditId valido. O ID pode aparecer em links, logs, respostas do historico global quando F-02 esta ativo ou compartilhamentos internos.

Evidencia - app/api/audit/chat/route.ts:114-125, 131-141, 168-183, 198-213

```
114: async function gravarAchadoNoParecer(auditId: string, report: AuditReport) {
119:   await prisma.audit.update({
120:     where: { id: auditId },
131:     export async function POST(request: Request) {
136:       await requireActor();
168:       const auditId = String(body.auditId ?? "");
183:       const memorias = auditId ? await carregarMemoriaDoDocumento(auditId) : [];
198:       historicoDaObra: () => historicoDaObra({ auditId, projectId: body.projectId ?? null }),
213:       if (auditId) await gravarAchadoNoParecer(auditId, atualizado);
```

Evidencia - lib/memoria-do-documento.ts:104-113

```
104: export async function carregarMemoriaDoDocumento(auditId: string) {
109:   const linhas = await prisma.auditText.findMany({
110:     where: { auditId },
111:     orderBy: { createdAt: "asc" },
112:     select: { fileName: true, pages: true, capitulos: true, charCount: true },
113:   });
```

F-04 - Endpoints auxiliares de auditoria usam ID sem validar organizacao

ALTA

Categoria: IDOR. Tres fluxos repetem o mesmo erro. O cancelamento altera qualquer auditoria PROCESSING pelo ID. Feedback lista e grava linhas de qualquer auditoria, inclusive notas, responsaveis e vereditos. Delta carrega a impressao de uma auditoria anterior apenas pelo ID. Todos autenticam, mas nenhum liga o objeto ao actor.organizationId ou ao dono legado.

Status de remediacao em 31/08/2026: CORRIGIDO NO WORKTREE - cancelamento, feedback e delta aplicam o mesmo predicado de posse por auditoria.

Condicao de explorabilidade: Exige sessao de membro e auditId valido. Cancelamento exige a janela em que status=PROCESSING. Feedback em producao exige NEXODOC_ENABLE_PUBLIC_AUDIT_HISTORY=true. Delta exige auditoria COMPLETED com impressao.

Evidencia - app/api/audit/fid/cancel/route.ts:58-64

```
58: const updated = await getPrisma().audit.updateMany({
59:   where: { id, status: "PROCESSING" },
60:   data: {
61:     status: "CANCELED",
62:     error: "Auditoria cancelada pelo usuario.",
63:     completedAt: new Date(),
64:   },
```

Evidencia - app/api/audits/[id]/feedback/route.ts:68-71, 234-237, 274-308

```

68: const feedback = await getPrisma().auditFeedback.findMany({
69:   where: { auditId: id },
70:   orderBy: { createdAt: "asc" },
71: });
234: const audit = await getPrisma().audit.findUnique({
235:   where: { id },
236:   select: { id: true },
237: });
274: const feedback = ... auditFeedback.create/upsert({ auditId: id, ... });

```

Evidencia - app/api/audit/delta/route.ts:63-65

```

63: const anterior = await getPrisma().audit.findFirst({
64:   where: { id: auditIdAnterior },
65:   select: { id: true, report: true, completedAt: true, createdAt: true, status: true },
66: });

```

F-05 - AuditLearning e global e pode ser alterado por membro de qualquer organizacao

MEDIA

Categoria: Banco sem tranca. O schema diz que o aprendizado e do escritorio, mas AuditLearning nao possui organizationId. GET/POST/PATCH/DELETE chamam requireActor e descartam o ator; as consultas e mutacoes sao globais. Esses textos entram no prompt de toda auditoria. Em uma instalacao com duas organizacoes, um membro le, altera ou exclui as regras da outra e pode influenciar os resultados de IA.

Status de remediacao em 31/08/2026: CORRIGIDO NO WORKTREE E NO BANCO DEV - organizationId e obrigatorio, o legado foi migrado para org-prosul e todas as operacoes usam o tenant. Regra de papel e autoria continuam como hardening.

Condicao de explorabilidade: Impacto entre tenants exige duas organizacoes na mesma base. Mesmo com uma organizacao, qualquer MEMBER ativo consegue alterar o acervo global por chamada direta.

Evidencia - prisma/schema.prisma:698-717

```

698: /// Sem relacao com `User`: aprendizado e do ESCRITORIO...
701: model AuditLearning {
702:   id String @id @default(cuid())
703:   title String
704:   content String
707:   type String @default("preference")
709:   scope String @default("global")
711:   status String @default("active")
717:   @@index([status, scope, updatedAt])
// Nao existe organizationId.

```

Evidencia - app/api/learnings/route.ts:49-63 66-88

```

49: export async function GET(request: Request) {
54:   await requireActor();
61:   const learnings = await listAuditLearnings();
66:   export async function POST(request: Request) {
71:   await requireActor();
80:   const learning = await createAuditLearning({ ... });

```

Evidencia - lib/audit-learnings.ts:205-215 235-246 280-303 337

```

205: const rows = await getPrisma().auditLearning.findMany({ where: { ... } });
235: const row = await getPrisma().auditLearning.create({ data: { ... } });
280: const atual = await getPrisma().auditLearning.findUnique({ where: { id } });
302: const row = await getPrisma().auditLearning.update({ where: { id }, ... });
337: const { count } = await getPrisma().auditLearning.deleteMany({ where: { id } });

```

F-06 - Criacao de artefato aceita auditId e IdDraftId de outro projeto ou tenant

MEDIA

Categoria: IDOR. A rota confirma acesso apenas ao projectId do caminho. Em seguida aceita auditId e IdDraftId do corpo e createDocumentArtifact grava as FKs diretamente, sem provar que os objetos relacionados pertencem ao mesmo projeto ou organizacao. Com IDs validos, um membro pode criar relacoes cruzadas e corromper a trilha de auditoria; respostas diferentes entre FK valida e invalida tambem funcionam como oraculo de existencia.

Status de remediacao em 31/08/2026: CORRIGIDO NO WORKTREE - auditId e IdDraftId sao validados contra o mesmo projectId antes da criacao; prova cruzada passou.

Condicao de explorabilidade: Exige acesso ao projeto do path e conhecimento de um auditId ou IdDraftId valido de outro contexto. A FK precisa existir; IDs inexistentes sao recusados pelo banco.

Evidencia - app/api/projects/Id/artifacts/route.ts:167-190

```
167: try {
168:   await assertProjectAccess(id, actor);
173:   const expiresAtValue = getStringField(payload?.expiresAt);
174:   const artifact = await getPrisma().$transaction((tx) =>
175:     createDocumentArtifact(tx, {
176:       projectId: id,
177:       auditId: getStringField(payload?.auditId) || undefined,
178:       IdDraftId: getStringField(payload?.IdDraftId) || undefined,
190:       metadata: payload?.metadata as Prisma.InputJsonValue | undefined,
```

Evidencia - lib/project-store.ts:231-248

```
231: const artifact = await tx.documentArtifact.create({
232:   data: {
233:     projectId: input.projectId ?? undefined,
234:     auditId: input.auditId ?? undefined,
235:     IdDraftId: input.IdDraftId ?? undefined,
236:     userId: input.actor.id ?? undefined,
237:     ...
248:     metadata: input.metadata,
```

F-07 - Chave OpenAI real foi copiada para teste e permanece no Git

ALTA

Categoria: Chaves expostas. Um literal com formato sk-proj aparece no teste que valida nomes de modelo; o proprio nome do teste diz que uma chave ja foi parar no banco. O valor foi introduzido no commit 311196f e continua no HEAD, portanto clones e forks preservam a credencial. A auditoria nao tentou autenticar com a chave. O literal nao foi encontrado em .next/static, entao nao ha evidencia de exposicao no bundle do navegador.

Status de remediacao em 31/08/2026: PARCIAL - literal removido e scanner local adicionado; revogacao e limpeza do historico remoto continuam pendentes.

Condicao de explorabilidade: Exploravel enquanto a chave estiver ativa e com permissao/quota. Se ja foi revogada, o risco operacional atual cai, mas o repositorio continua contendo material secreto historico.

Evidencia - scripts/test-ai-precos.ts:90-94

```
90: test("nome de modelo nao aceita chave de API - uma ja foi parar no banco", () => {
91:   const chave = "sk-proj-4Ek...REDACTED";
92:   assert.notEqual(validateAiModelName(chave), "");
93:   assert.notEqual(validateAiModelName(" sk-abc123 "), "");
94:   assert.notEqual(validateAiModelName("SK-PROJ-MAIUSCULO"), "");
```

Evidencia - Historico Git

```
git log -S<valor> -- scripts/test-ai-precos.ts
311196f custo: o painel para de somar zero pelo que nao tem preco, e chave de API deixa de passar por modelo
```

5. Pontos fortes com evidencia

Controle	Evidencia verificada
Cobertura de autenticacao	Foram enumerados 54 arquivos de rota e 74 handlers. A prova automatizada reconheceu portao em todos; apenas /api/auth/[...nextauth] e /api/saude sao excecoes publicas intencionais.
Admin no servidor	Todos os oito arquivos sob app/api/admin usam checkAdminRequest. O gate exige sessao de administrador de plataforma e Bearer NEXODOC_ADMIN_TOKEN; a pagina tambem redireciona nao-admins.
Projetos e sub-recursos	A listagem usa organizationId derivado do ator. assertProjectAccess exige membership ACTIVE e e chamado antes das consultas de documentos, uploads, artefatos e volume.
Conversas do Nexo	GET e DELETE por ID filtram simultaneamente id e userEmail; o PUT recusa colisao de UUID pertencente a outro usuario.
Leitura principal de uma auditoria	GET /api/audits/[id] usa project.organizationId e conserva legado somente para userId do proprio ator, evitando casar null com null.
Permissoes espelhadas	Os gates de interface para /admin correspondem ao gate servidor. Convite de membro e criacao manual de projeto tambem verificam orgRole/isPlatformAdmin no backend.
XSS	Nao ha dangerouslySetInnerHTML, innerHTML, eval, new Function ou parser Markdown/HTML em codigo de producao. React faz escape de texto. E-mails aplicam escaping explicito em campos dinamicos.
Segredos e deploy	.env.local esta ignorado; variaveis secretas em .env.example e render.yaml nao possuem valor hardcoded; migracao de producao falha se DATABASE_URL faltar; a chave encontrada nao aparece em .next/static.
Dev auth	O provider de credenciais exige NODE_ENV diferente de production e NEXODOC_DEV_AUTH=true; render.yaml fixa a flag como false.
Escopo central de auditoria	lib/audit-access.ts concentra organizacao e legado por userId. A prova com dois escritorios cobre historico, qualidade, feedback, cancelamento, delta e chat.

Testes executados

```

npm run test:security:audits -> OK, 6 regras + 1.174 arquivos escaneados
npm run prova:security:audits -> OK, 12 cenarios entre dois escritorios
npm run test:security:p2 -> OK, 5 regressões estruturais
npx prisma migrate deploy -> OK, migration de AuditLearning aplicada
npm run prova:security:p2 -> OK, 5 provas reais com banco
npx tsc --noEmit -> OK
ESLint dos arquivos alterados -> OK

```

As provas criam dois escritorios temporarios. O P1 tenta listar, ler/gravar feedback, cancelar, comparar e abrir o chat com auditId estrangeiro. O P2 tenta ler, alterar e excluir aprendizados alheios e relacionar auditorias ou rascunhos de outro projeto; todos os dados de prova sao removidos ao final.

6. Cobertura sistematica dos handlers

A tabela resume todos os 54 arquivos de rota. Grupos marcados como corretos foram inspecionados quanto a autenticação, fonte do tenant e IDs de path/query/body. Os detalhes dos grupos com falha remetem aos achados.

Arquivo/grupo	Resultado	Evidencia de cobertura
app/api/admin/* (8 arquivos, 14 handlers)	Correto	checkAdminRequest em todas as rotas; sessao admin + token.
app/api/projects/route.ts	Correto	GET filtra organizationId; POST exige coordenacao e ignora organizationId do corpo.
app/api/projects/[id]/route.ts	Correto	GET/PATCH/DELETE validam membership ACTIVE antes de operar.
app/api/projects/[id]/documents/route.ts	Correto	GET/POST chamam assertProjectAccess.
app/api/projects/[id]/uploads/route.ts	Correto	GET/POST chamam assertProjectAccess.
app/api/projects/[id]/artifacts/route.ts	Corrigido P2	Projeto e validado; auditId/IdDraftId tambem precisam pertencer ao mesmo projectId.
app/api/audits/[id]/route.ts	Correto	Escopo por organizacao; legado por userId do ator.
app/api/audits/[id]/atribuir + avisar	Correto	Helpers validam audit.project.organizationId e destinatario do mesmo tenant.
app/api/audit/[id]/cancel/route.ts	Corrigido P1	Update combina id/status com auditByIdWhereForActor.
app/api/audits/[id]/feedback/route.ts	Corrigido P1	GET/POST exigem auditoria no escopo; nomes tambem ficam no escritorio.
app/api/audit/delta/route.ts	Corrigido P1	Auditoria anterior usa auditByIdWhereForActor.
app/api/audit/chat/route.ts	Corrigido P1	Memoria e update so ocorrem apos posse; projectId e derivado no servidor.
app/api/audits/recent + quality	Corrigido P1	Consultas usam auditWhereForActor; qualidade filtra pela relacao Audit.
app/api/audit/route.ts	Correto	Exige ator, projectId e assertProjectAccess; base anterior limitada ao mesmo projectId.
app/api/nexo/conversas + [id]	Correto	Lista e operacoes por ID amarradas ao e-mail da sessao.
app/api/nexo/usage/route.ts	Correto	conversationId combinado com userEmail.
app/api/organizacao/membros/route.ts	Correto	GET escopado; POST exige coordenacao e usa organizationId do ator.
app/api/learnings + [id]	Corrigido P2	organizationId do ator acompanha GET/POST/PATCH/DELETE e todas as consultas ao storage.
app/api/volume/* (5 arquivos)	Correto	Portao presente; persistencia por projeto chama assertProjectAccess.
app/api/nexo/* restante, capas, LD, painel, trabalho	Correto	Sem lookup de objeto arbitrario ou consultas escopadas por ator/helper.
app/api/auth/[...nextauth] + app/api/saude	Excecao	Entrada do Auth.js e health check publicos por desenho.

7. Recomendacoes priorizadas

Prioridade	Acao	Resultado esperado
P1	Fechar o autoingresso	Definir NEXODOC_ESCRITORIO_PADRAO vazio no deploy e alterar o default do codigo para convite/allowlist.
P1 feito	Centralizar acesso a auditoria	Helper criado e aplicado no chat, cancelamento, feedback, delta, leitura individual e updates.
P1 parcial	Rotacionar a chave	Fixture e HEAD corrigidos; ainda revogar no provedor, revisar uso/custo e limpar o historico Git coordenadamente.
P1 feito	Escopar consultas globais	recent/quality usam o ator; prova com duas organizacoes passou.
P2 feito	Tenant em AuditLearning	organizationId obrigatorio, legado migrado para org-prosul e consultas/mutacoes isoladas por escritorio.
P2 feito	Validar relacoes de artefato	auditId/IdDraftId precisam pertencer ao mesmo projectId; combinacoes cruzadas sao recusadas.
P2 hardening	Governanca dos aprendizados	Definir quem pode mutar o acervo por papel e registrar autor/trilha de alteracao.
P2	Testes de matriz	Para cada rota com ID, testar: mesmo tenant, outro tenant, objeto inexistente, legado e usuario desativado.
P3	Automacao preventiva	Adicionar secret scanning e uma regra estatica que sinalize Prisma por ID em route.ts sem helper de acesso.
P3	Revisao de politica	Documentar capacidades por papel e manter a verificacao no servidor como fonte de verdade.

Ordem sugerida de execucao

O isolamento de auditorias, aprendizados e artefatos ja foi aplicado e provado. A ordem restante e: fechar a entrada automatica, revogar a chave, revisar uso/custo, coordenar a limpeza do historico Git e definir governanca de mutacao/autoria dos aprendizados.

ISSUES PARA O GITHUB

Blocos completos em Markdown, prontos para copiar e colar ou registrar a remediacao. Issues 2 a 6 ja estao corrigidas no worktree; a migracao da issue 5 tambem foi aplicada no banco de desenvolvimento. A issue 7 esta parcial e a issue 1 continua aberta. Achados auxiliares relacionados foram agrupados para evitar spam.

Issue 1

--- ISSUE 1 ---

Titulo: [Seguranca] Remover ingresso automatico de qualquer conta Google no escritorio padrao
Labels sugeridas: security, critica

Descricao do problema e por que e exploravel

O login Google nao possui allowlist de dominio/e-mail e, no primeiro acesso, `getUserAccess` chama `garantirEscritorioPadrao`. Quando `NEXODOC\_ESCRITORIO\_PADRAO` esta ausente, `escritorioPadrao()` retorna `org-prosul` e o sistema cria um `OrganizationMember` com `role: MEMBER` e `status: ACTIVE`. O `render.yaml` atual nao declara a variavel, portanto o comportamento padrao esta ativo no deploy descrito pelo repositorio. Uma conta Google externa passa a ser membro e recebe o mesmo acesso por organizacao concedido a colaboradores.

Evidencia

- `auth.ts:11-36`: provider Google sem allowlist de conta ou dominio.
- `lib/access-control.ts:121-122`: todo login chama `garantirEscritorioPadrao`.
- `lib/access-control.ts:255-264`: cria o vinculo `MEMBER/ACTIVE`.
- `lib/access-control.ts:274-279`: variavel ausente retorna `org-prosul`.

Impacto

Leitura e uso indevidos dos projetos, auditorias, documentos derivados e funcoes de IA da organizacao; possibilidade de combinar o acesso com os IDORs descritos nas outras issues.

Sugestao de correcao

Fazer o padrao ser fechado: variavel ausente deve significar "sem autoingresso". Exigir convite pendente ou allowlist corporativa antes de criar membership. Declarar explicitamente `NEXODOC\_ESCRITORIO\_PADRAO=` no deploy enquanto a migracao nao for publicada. Considerar restringir o provider Google por dominio verificado, sem usar `hd` apenas como prova de autorizacao.

Critérios de aceite

- [] Conta Google sem convite recebe 403 e nenhum OrganizationMember e criado.
- [] Convite pendente vira ACTIVE no primeiro login e preserva o papel escolhido.
- [] Conta DISABLED nao e reativada por novo login.
- [] Ausencia e string vazia de NEXODOC_ESCRITORIO_PADRAO resultam em modo fechado.
- [] Teste de integracao cobre conta externa, convidada e desativada.
- [] Deploy de producao declara explicitamente a politica adotada.

--- FIM ISSUE 1 ---

Issue 2

--- ISSUE 2 ---

Titulo: [Seguranca] Escopar historico recente e metricas de qualidade pela organizacao do ator
Labels sugeridas: security, alta

Descricao do problema e por que e exploravel

`GET /api/audits/recent` autentica o ator, mas executa `audit.findMany` sem `where` de organizacao e devolve IDs, `result`, `report` e nomes de arquivos de toda a base. `GET /api/audits/quality` faz `groupBy` global de feedback. Em producao os endpoints dependem de `NEXODOC\_ENABLE\_PUBLIC\_AUDIT\_HISTORY=true`; em ambientes nao-production ficam ativos automaticamente. Com duas organizacoes, um membro de uma ve dados da outra.

Evidencia

- `app/api/audits/recent/route.ts:10-14,58-85`: feature flag e consulta sem filtro de tenant.
- `app/api/audits/quality/route.ts:31-34`: agregacao global.

Impacto

Exposicao de pareceres completos, resultados, nomes de arquivo e IDs de auditoria; mistura de indicadores de qualidade entre clientes.

Sugestao de correcao

Capturar o retorno de `requireActor()` e aplicar `where: { project: { organizationId: actor.organizationId } }`. Tratar auditorias legadas sem projeto apenas pelo `userId` do ator, reproduzindo a regra de `GET /api/audits/[id]`. Nas metricas, filtrar via relacao `audit.project.organizationId` antes de agrupar.

Critérios de aceite

- [] Membro da organizacao A nao recebe auditorias da organizacao B.
- [] IDs, reports, results e nomes de arquivo de B nunca aparecem na resposta de A.
- [] Qualidade de A nao inclui feedback de B.
- [] Legado sem projeto so aparece ao proprio autor.
- [] Testes cobrem duas organizacoes e a feature flag ligada.

--- FIM ISSUE 2 ---

Issue 3

--- ISSUE 3 ---

Titulo: [Seguranca] Validar posse da auditoria antes de carregar memoria ou gravar achados no chat
Labels sugeridas: security, alta

Descricao do problema e por que e exploravel

`POST /api/audit/chat` exige apenas um membro ativo. O `auditId` vem do corpo e `carregarMemoriaDoDocumento` consulta `AuditText` somente por esse ID. O modelo pode responder perguntas sobre o texto integral. Ao registrar um achado, a rota atualiza `Audit` pelo mesmo ID sem organizacao. O `projectId` usado pelo historico tambem vem do cliente. Quem conhece um ID alheio pode exfiltrar o memorial por linguagem natural e modificar o parecer persistido.

Evidencia

- `app/api/audit/chat/route.ts:114-125`: update por `id` apenas.
- `app/api/audit/chat/route.ts:131-141,168-183`: ator autenticado, ID do corpo e carga de memoria.
- `app/api/audit/chat/route.ts:198-213`: projectId do corpo e gravacao pelo auditId.
- `lib/memoria-do-documento.ts:104-113`: consulta de texto apenas por auditId.

Impacto

Confidencialidade do texto integral de documentos e integridade do parecer/contagem de achados de outra organizacao.

Sugestao de correcao

Resolver a auditoria uma unica vez por `id + organizationId` (ou legado por userId) antes de iniciar SSE. Derivar projectId do registro autorizado, nunca do corpo. Passar um objeto de escopo/autorizacao aos helpers e incluir `project.organizationId` nos updates, preferencialmente em transacao.

Critérios de aceite

- [] AuditId de outra organizacao retorna 404 antes de chamar IA.
- [] Nenhum AuditText alheio e carregado.
- [] projectId do historico vem da auditoria autorizada.
- [] Update do parecer inclui o mesmo predicado de acesso.
- [] Teste prova leitura e escrita negadas entre duas organizacoes.

--- FIM ISSUE 3 ---

Issue 4

--- ISSUE 4 ---

Titulo: [Seguranca] Centralizar autorizacao por auditId nos endpoints auxiliares

Labels sugeridas: security, alta

Descricao do problema e por que e exploravel

Cancelamento, feedback e delta autenticam o usuario, mas operam por `auditId` sem verificar `project.organizationId` ou o dono legado. Um membro com um ID alheio pode cancelar uma auditoria em processamento; quando o historico publico esta habilitado, pode ler e sobrescrever feedback; e pode consultar metadados/impressao usados pelo delta.

Evidencia

- `app/api/audit/[id]/cancel/route.ts:58-64`: update por id/status.
- `app/api/audits/[id]/feedback/route.ts:68-71,234-237,274-308`: leitura e upsert por auditId.
- `app/api/audit/delta/route.ts:63-65`: busca da base por ID apenas.

Impacto

Negacao de servico direcionada, adulteracao de vereditos/notas/resolucao e leitura de metadados de auditorias alheias.

Sugestao de correcao

Criar um helper unico `findAuthorizedAudit(actor, id)` com a mesma regra de `GET /api/audits/[id]`. Usar o ID retornado pelo helper em todas as queries e, para escrita concorrente, repetir o predicado de organizacao no `updateMany`/transacao.

Critérios de aceite

- [] Cancelamento alheio retorna 404 e nao muda status.
- [] GET/POST de feedback alheio retornam 404 e nao criam linha.
- [] Delta alheio retorna resposta indistinguivel de ID inexistente.
- [] Legado sem projeto continua acessivel somente ao proprio autor.
- [] Testes cobrem PROCESSING, COMPLETED e feature flag de feedback.

--- FIM ISSUE 4 ---

Issue 5

--- ISSUE 5 ---

Titulo: [Seguranca] Tornar AuditLearning pertencente a uma organizacao e restringir mutacoes
Labels sugeridas: security, media

Descricao do problema e por que e exploravel

`AuditLearning` e descrito como conhecimento do escritorio, mas nao possui `organizationId`. As rotas descartam o ator depois de autenticar e todas as leituras/escritas sao globais. O conteudo ativo entra no prompt de toda auditoria. Em uma base multi-tenant, um membro pode ler, alterar ou excluir regras de outro escritorio e influenciar resultados de IA.

Evidencia

- `prisma/schema.prisma:698-717`: modelo sem organizationId.
- `app/api/learnings/route.ts:49-63,66-88`: ator nao usado para escopo ou papel.
- `app/api/learnings/[id]/route.ts:49-67,89-105`: PATCH/DELETE por ID global.
- `lib/audit-learnings.ts:205-215,235-246,280-303,337`: consultas e mutacoes globais.

Impacto

Vazamento e destruicao de conhecimento interno; contaminacao de prompts e resultados de auditoria entre organizacoes.

Sugestao de correcao

Adicionar `organizationId` obrigatorio, migrar registros atuais para a organizacao correta e incluir o tenant em todas as chaves/queries. Definir explicitamente quem pode mutar o acervo (por exemplo OWNER/ADMIN) e aplicar o gate no servidor. Registrar autor e trilha de alteracao.

Critérios de aceite

- [] AuditLearning possui organizationId com FK e indice.
- [] Listagem e prompt recebem apenas aprendizados da organizacao do ator.
- [] PATCH/DELETE incluem id + organizationId.
- [] Papel sem permissao de configuracao recebe 403 em POST/PATCH/DELETE.
- [] Migracao nao perde os aprendizados atuais.
- [] Testes cobrem duas organizacoes e tentativa de prompt poisoning cruzado.

--- FIM ISSUE 5 ---

Issue 6

--- ISSUE 6 ---

Titulo: [Seguranca] Validar auditId e ldDraftId ao registrar artefatos de projeto
Labels sugeridas: security, media

Descricao do problema e por que e exploravel

`POST /api/projects/[id]/artifacts` valida acesso ao projeto do path, mas aceita `auditId` e `ldDraftId` do corpo e grava as FKs diretamente. Nao ha prova de que essas entidades pertencem ao mesmo projeto/organizacao. Com um ID valido alheio, o usuario cria uma relacao cruzada e pode usar o sucesso/erro de FK como oraculo de existencia.

Evidencia

- `app/api/projects/[id]/artifacts/route.ts:167-190`: acesso somente ao projectId; IDs relacionados saem do payload.
- `lib/project-store.ts:231-248`: FKs gravadas sem validacao adicional.

Impacto

Corrupcao da trilha de documentos e auditorias, relacoes cruzadas entre tenants e confirmacao de existencia de IDs.

Sugestao de correcao

Quando auditId/ldDraftId forem informados, buscar os objetos com `projectId: id` e com o tenant do ator antes do create.

Rejeitar combinacoes sem projeto ou inconsistentes com 400/404 uniforme. Se a API nao precisa aceitar essas relacoes do cliente, removê-las do payload e deriva-las no servidor.

Critérios de aceite

- [] auditId de outro projeto/tenant e recusado sem criar artefato.
- [] ldDraftId de outro projeto/tenant e recusado sem criar artefato.
- [] IDs inexistentes e alheios produzem resposta indistinguivel.
- [] IDs do mesmo projeto continuam funcionando.
- [] Testes verificam ausencia de relacoes cruzadas no banco.

--- FIM ISSUE 6 ---

Issue 7

--- ISSUE 7 ---

Titulo: [Seguranca] Revogar e remover chave OpenAI copiada para teste e historico Git

Labels sugeridas: security, alta

Descricao do problema e por que e exploravel

`scripts/test-ai-precios.ts` contem um literal `sk-proj-...` com aparencia de chave real; o proprio teste registra que uma chave ja foi parar no banco. O valor entrou no commit `311196f` e permanece no HEAD. Mesmo que o arquivo seja corrigido, clones e forks mantem o segredo no historico. A auditoria nao tentou usar a credencial e nao a encontrou no bundle `.next/static`.

Evidencia

- `scripts/test-ai-precios.ts:90-94`: literal hardcoded (redigido no relatorio como `sk-proj-4Ek...REDACTED`).
- `git log -S<valor> -- scripts/test-ai-precios.ts`: commit introdutor `311196f`.

Impacto

Uso indevido de API, consumo de quota/custo e acesso ao projeto associado conforme as permissoes da chave.

Sugestao de correcao

Revogar/rotacionar a chave no provedor imediatamente; substituir o teste por uma string obviamente ficticia; investigar uso e cobranca desde 13/08/2026. Reescrever o historico com `git filter-repo` ou ferramenta equivalente e coordenar force-push/reclone. Adicionar secret scanning no CI e protecao de push.

Critérios de aceite

- [] Chave antiga consta como revogada no provedor.
 - [] Logs de uso e cobranca foram revisados e incidentes documentados.
 - [] Nenhum `sk-proj-` real existe no HEAD nem no historico remoto oficial.
 - [] Teste usa apenas placeholder inequivocamente falso.
 - [] Scanner de segredos bloqueia novo commit com chave valida.
 - [] Colaboradores foram orientados a reclonar apos a limpeza do historico.
- FIM ISSUE 7 ---